Smart Campus Management System

CS-104L Object-Oriented Programming

HITEC University Taxila

Date: 05-06-2026

GitHub URL: https://github.com/ahmadoffcl/OOP

Group Members

- Ahmad Ali | Reg No: 25-CS-067
- Umer Altaf | Reg No: 25-CS-057
- Muhammed Ahmad | Reg No: 25-CS-252

Introduction

This project is a simple command-line Smart Campus Management System written in C++. It manages basic campus records for students, faculty, courses, library items, fee records, hostel rooms, and reports. The purpose is to show all major OOP concepts from the CS-104L course in one connected program. The program uses simple arrays, pointers, classes, file handling, and exception handling. The design is intentionally easy to read so each group member can explain the code during viva.

System Modules

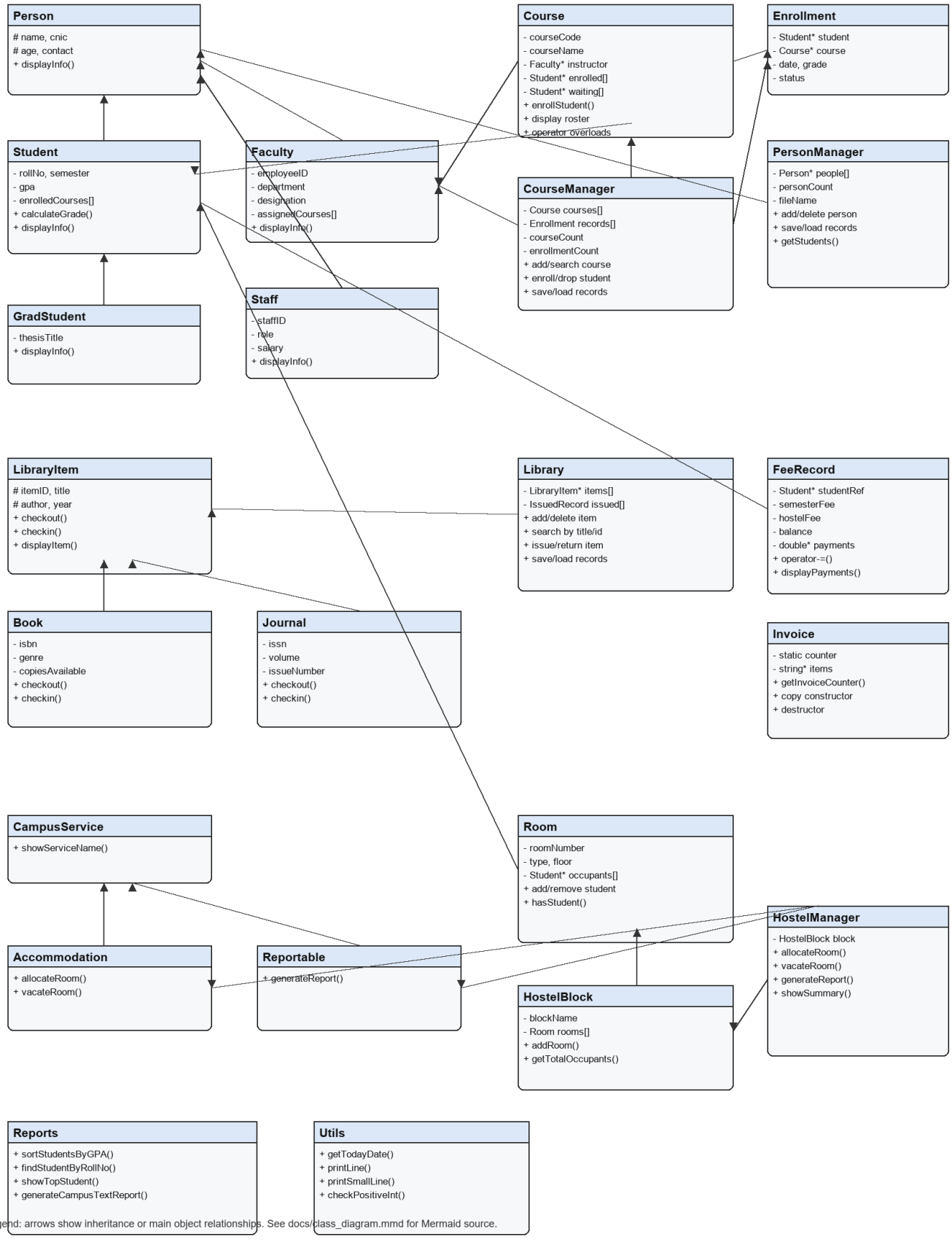
- Person module: Person, Student, GradStudent, Faculty, and Staff classes.
- PersonManager: saved person records with add, view, delete, save, and reload options.
- Course module: Course, Enrollment, and CourseManager classes with saved course/enrollment records and capacity checking.
- Library module: Book and Journal catalog with add/search/delete, issue/return, overdue fine, file handling, and arrays.
- Finance module: FeeRecord and Invoice classes with copy handling and static invoice counter.
- Hostel module: Room, HostelBlock, and HostelManager using composition and multiple inheritance.
- Reports module: student sorting, searching, campus text report, and PDF-style text report generation.

Class Diagram

The class diagram below shows the main classes and relationships used in the project. Inheritance is used in Person, LibraryItem, and hostel interfaces. Composition is used where HostelManager contains HostelBlock, and aggregation is used where Course stores a Faculty pointer without owning the faculty object.

Smart Campus Management System - Class Diagram

HITEC University Taxila | CS-104L OOP



Legend: arrows show inheritance or main object relationships. See docs/class_diagram.mmd for Mermaid source.

OOP Concepts Implementation

1. Classes and Objects: All modules define classes and make objects in main.cpp.
   `Student s1(...); Course oop(...);`
2. Encapsulation: Private data is changed through public getters, setters, and member functions.
   `string getRollNo() const; void setGPA(double newGPA);`
3. Default Constructor: Several classes provide simple empty/default object creation.
   `Student::Student() : Person(), semester(1), gpa(0.0) {}`
4. Parameterized Constructor: Objects can be created with real starting data.
   `Course::Course(string code, string name, int credits, Faculty* teacher, int capacity)`
5. Copy Constructor: Person, Course, FeeRecord, and Invoice copy object data.
   `FeeRecord::FeeRecord(const FeeRecord& other)`
6. Destructors: Library, Invoice, and HostelManager have destructors for cleanup.
   `Invoice::~Invoice() { delete[] items; }`
7. Single Inheritance: Student reuses common Person data and behavior.
   `class Student : public Person`
8. Multi-level Inheritance: GradStudent extends Student, which already extends Person.
   `class GradStudent : public Student`
9. Multiple Inheritance: HostelManager implements accommodation and report behavior.
   `class HostelManager : public Accommodation, public Reportable`
10. Virtual Inheritance: Accommodation and Reportable virtually inherit CampusService to avoid duplicate base parts.
    `class Accommodation : virtual public CampusService`
11. Abstract Classes: Person, LibraryItem, Accommodation, and Reportable cannot be directly instantiated.
    `virtual void displayInfo() const = 0;`
12. Runtime Polymorphism: Base pointers call the correct derived displayInfo function.
    `Person* people[MAX_PEOPLE]; people[i]->displayInfo();`
13. Operator Overloading ==: Course objects are compared by course code.
    `bool Course::operator==(const Course& other) const`
14. Operator Overloading <<: Course and Invoice can be printed using stream insertion.
    `ostream& operator<<(ostream& out, const Course& course)`
15. Operator Overloading +: Two course waiting lists are merged.
    `Student** Course::operator+(Course& other)`
16. Operator Overloading -=: FeeRecord records a payment and reduces balance.
    `fee -= 25000;`
17. Friend Functions: operator<< is declared as a friend for Course and Invoice.
    `friend ostream& operator<<(ostream& out, const Invoice& invoice);`
18. Static Members: Invoice has one shared invoiceCounter for all invoice objects.
    `int Invoice::invoiceCounter = 0;`
19. Deep Copy: FeeRecord and Invoice copy dynamic arrays instead of sharing them.
    `payments = new double[paymentCount];`

20. Search Functions: Library searches with a loop, and Reports uses std::find_if for roll number search.

```
Student* Reports::findStudentByRollNo(Student* students[], int count, string rollNo)
```

21. Array-based Collections: The project uses arrays for people, courses, library items, rooms, and students.

```
Person* people[MAX_PEOPLE];
```

22. Arrays of Objects: HostelBlock keeps Room objects in an array.

```
Room rooms[MAX_BLOCK_ROOMS];
```

23. Exception Handling: Custom exceptions are thrown and caught for capacity and overdue cases.

```
throw CapacityExceededException("Course is full");
```

24. File I/O: Person, course, enrollment, library, and report data are loaded/saved using fstream.

```
ofstream file(fileName);
```

25. Reporting and Utilities: Reports and Utils keep report, date, formatting, and validation helpers separate.

```
Reports::generateCampusTextReport(...);
```

26. Memory Management: Objects and arrays created with new are deleted using delete or delete[].

```
delete[] sortedStudents;
```

27. Sorting and Searching: Reports sorts students by GPA and searches by roll number.

```
sortStudentsByGPA(students, count);
```

28. Composition: HostelManager owns a HostelBlock object.

```
HostelBlock block;
```

29. Aggregation: Course stores Faculty* and Room stores Student* without owning those people.

```
Faculty* instructor; Student* occupants[];
```

Module Descriptions

- Module 1 - Person Hierarchy: Stores common personal information in Person and uses Student, GradStudent, Faculty, and Staff subclasses. PersonManager stores saved records, supports input, deletion, save, and reload.
- Module 2 - Course and Enrollment: Stores saved course data, enrolls saved students, tracks waiting-list records, saves/reloads text files, blocks over-capacity enrollment, and demonstrates overloaded operators.
- Module 3 - Library System: Stores books and journals, adds/deletes catalog items, searches by title or ID, saves/loads catalog and issued records, tracks issued items, and handles overdue fines.
- Module 4 - Fee and Finance: Stores fee balance, records payments with operator-=, deep-copies fee data, and generates invoices with a static counter.
- Module 5 - Hostel Management: Uses rooms, hostel blocks, multiple inheritance, virtual inheritance, and composition to allocate/vacate rooms.
- Module 6 - Reporting and Utilities: Sorts and searches student data, prints reports, writes a campus text report, writes a PDF-style text report, and keeps helper functions separate.

GitHub Workflow

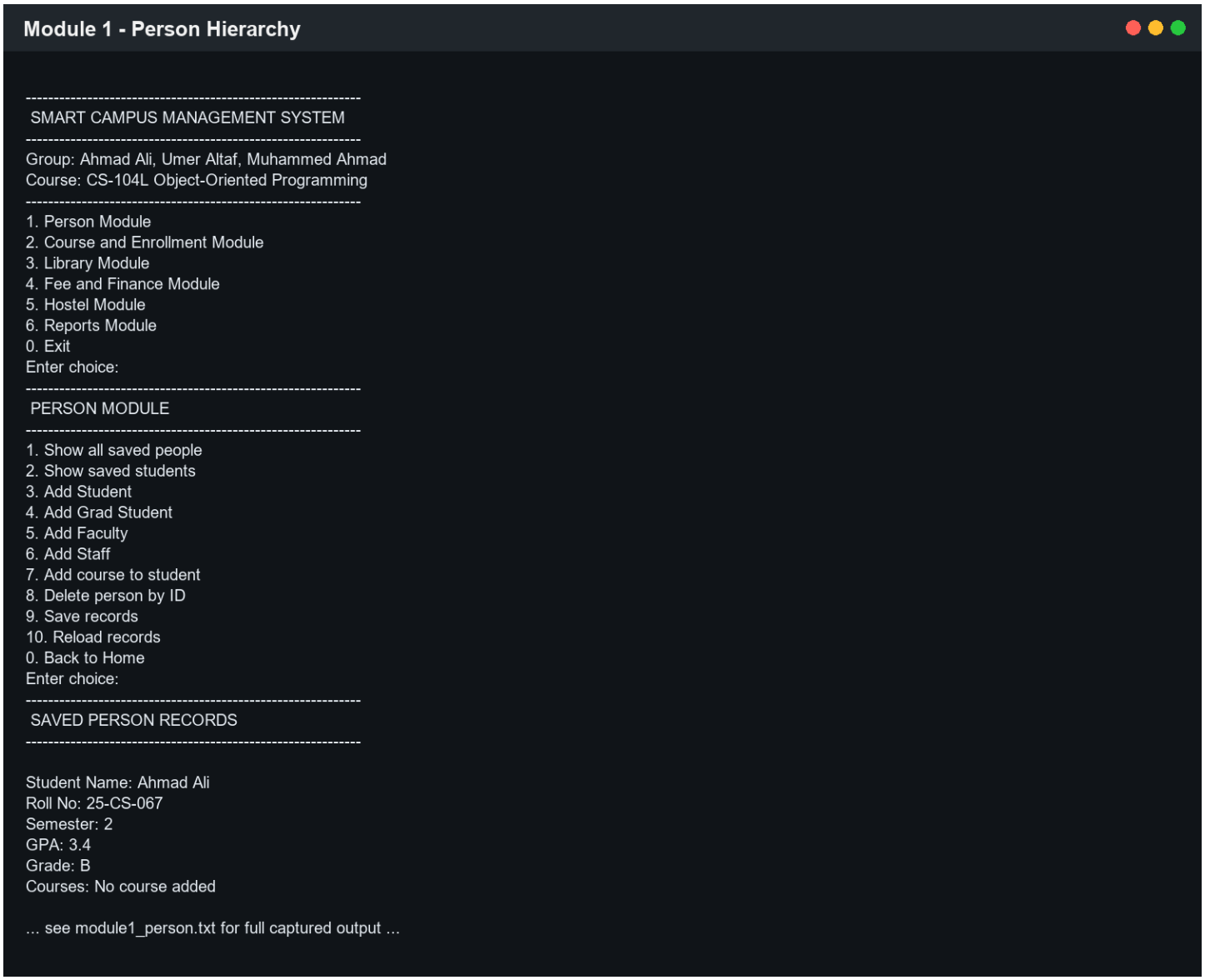
The project is already a local git repository with separate commits for setup, documentation, and each major module. Before final submission, create a public GitHub repository named HITEC-OOP-SCMS-25-CS-067, push this folder, and paste the repository URL here and in README.md.

- Local branch: master
- Current local commits: see git log --oneline in the repository.
- A GitHub Actions build workflow is included at .github/workflows/build.yml for the bonus compile check.
- Do not submit until the public GitHub link is shared through the course portal.

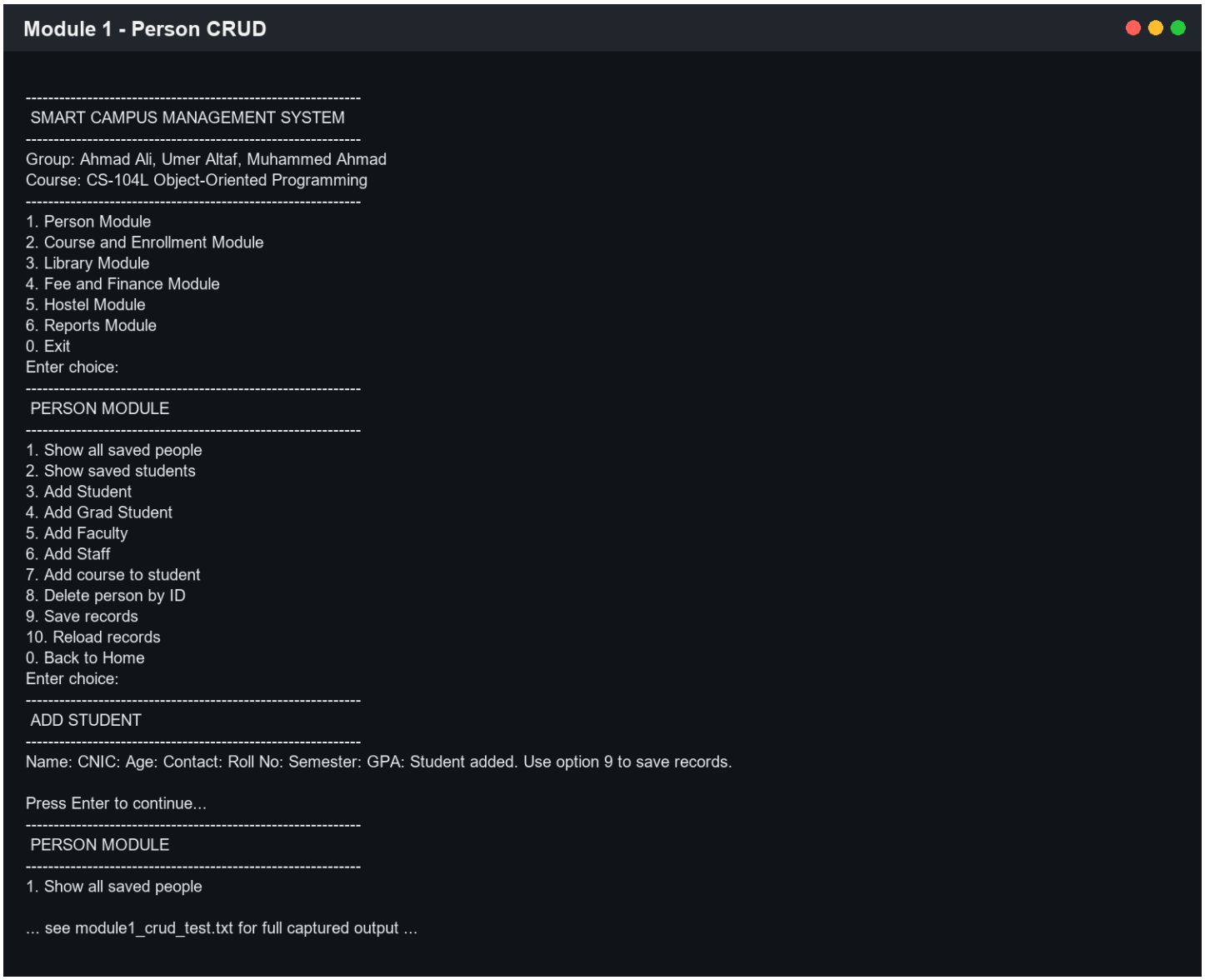
Testing Summary

- Build command .\build.bat completed successfully.
- All six home menu modules were tested.
- Screen-based navigation was tested with Enter pauses after action pages.
- Module 1 CRUD was tested: add, display, save, reload, delete, save, and reload.
- Module 1 add-course action was tested by adding CS-200 to 25-CS-067.
- Module 2 added a course, searched it, enrolled students, sent a full-course student to waiting list, showed roster, dropped a student, saved/reloaded records, compared courses, and merged waiting lists.
- Library module added book/journal records, searched by title and ID, issued and returned items, blocked duplicate issue, showed overdue fine, deleted an item, and saved/reloaded records.
- Finance demo showed payment, copy constructor, copy assignment, invoice, and invoice copy.
- Hostel demo showed service name, allocation, duplicate check, summary, report, and vacate room.
- Reports demo sorted students by GPA and created data/campus_report.txt.
- Reports demo created data/campus_pdf_report.txt.
- Wrong input test with abc did not freeze the program.
- Full captured output files are saved in docs/test_outputs.

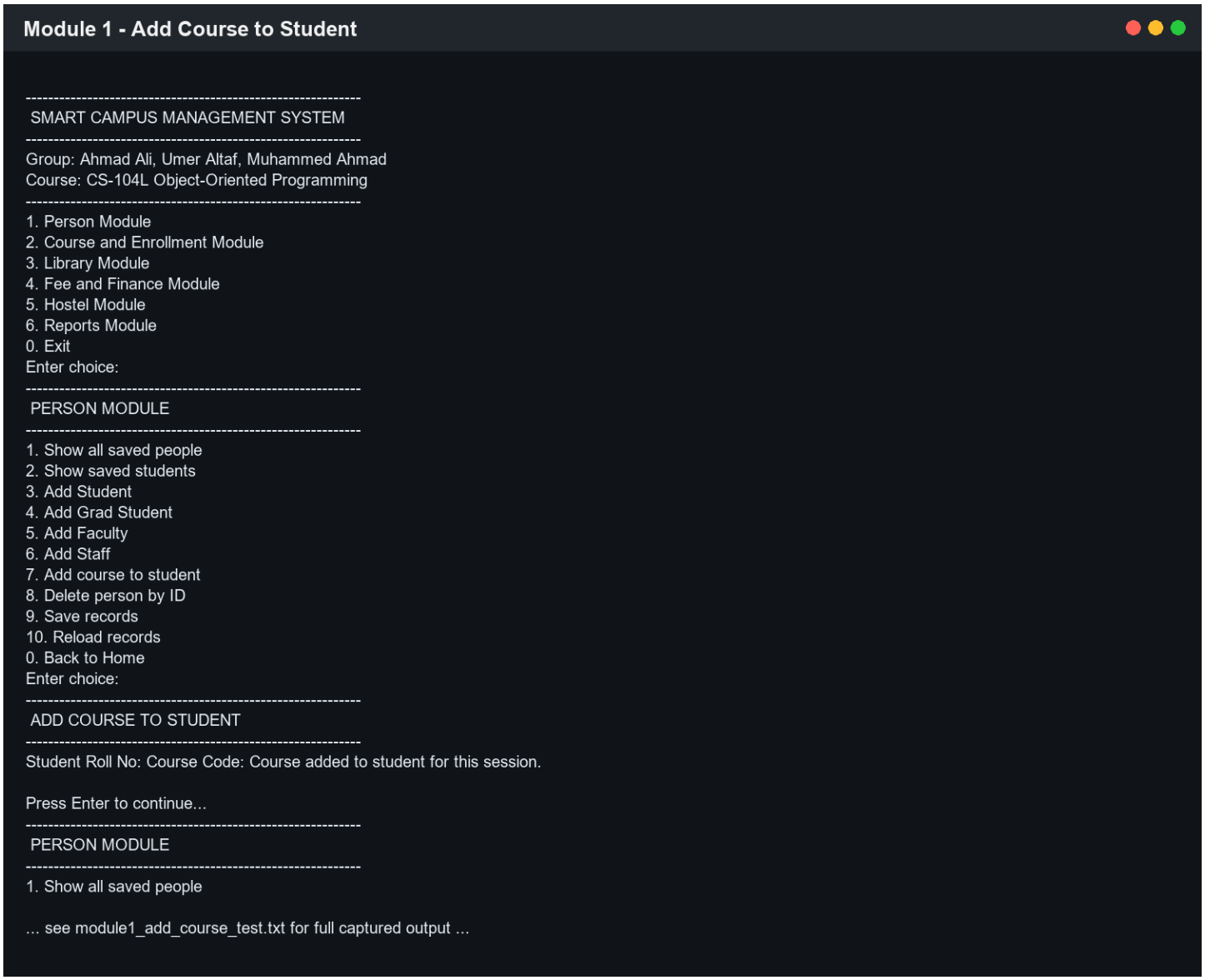
Module1 Person Output Screenshot



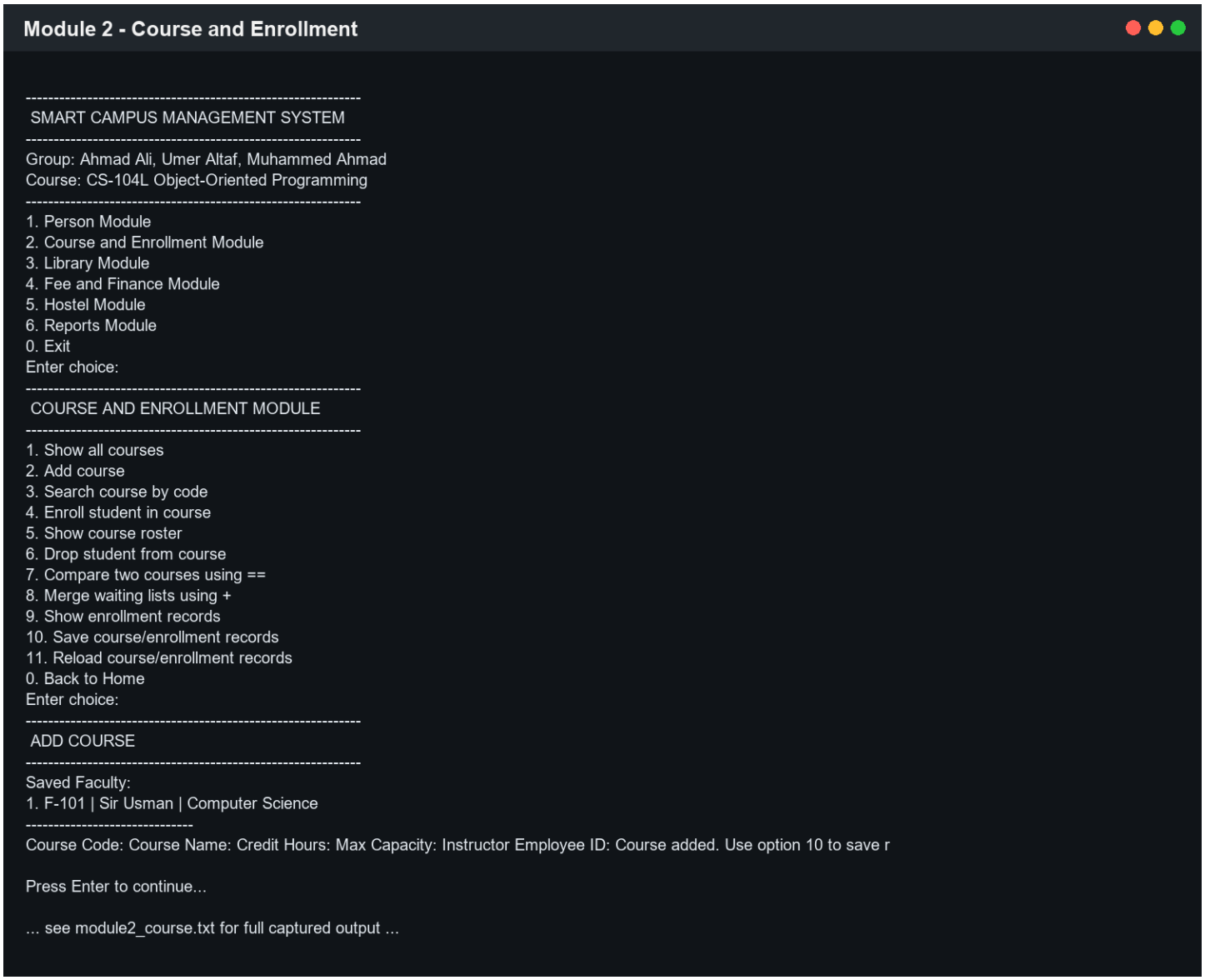
Module1 Crud Test Output Screenshot



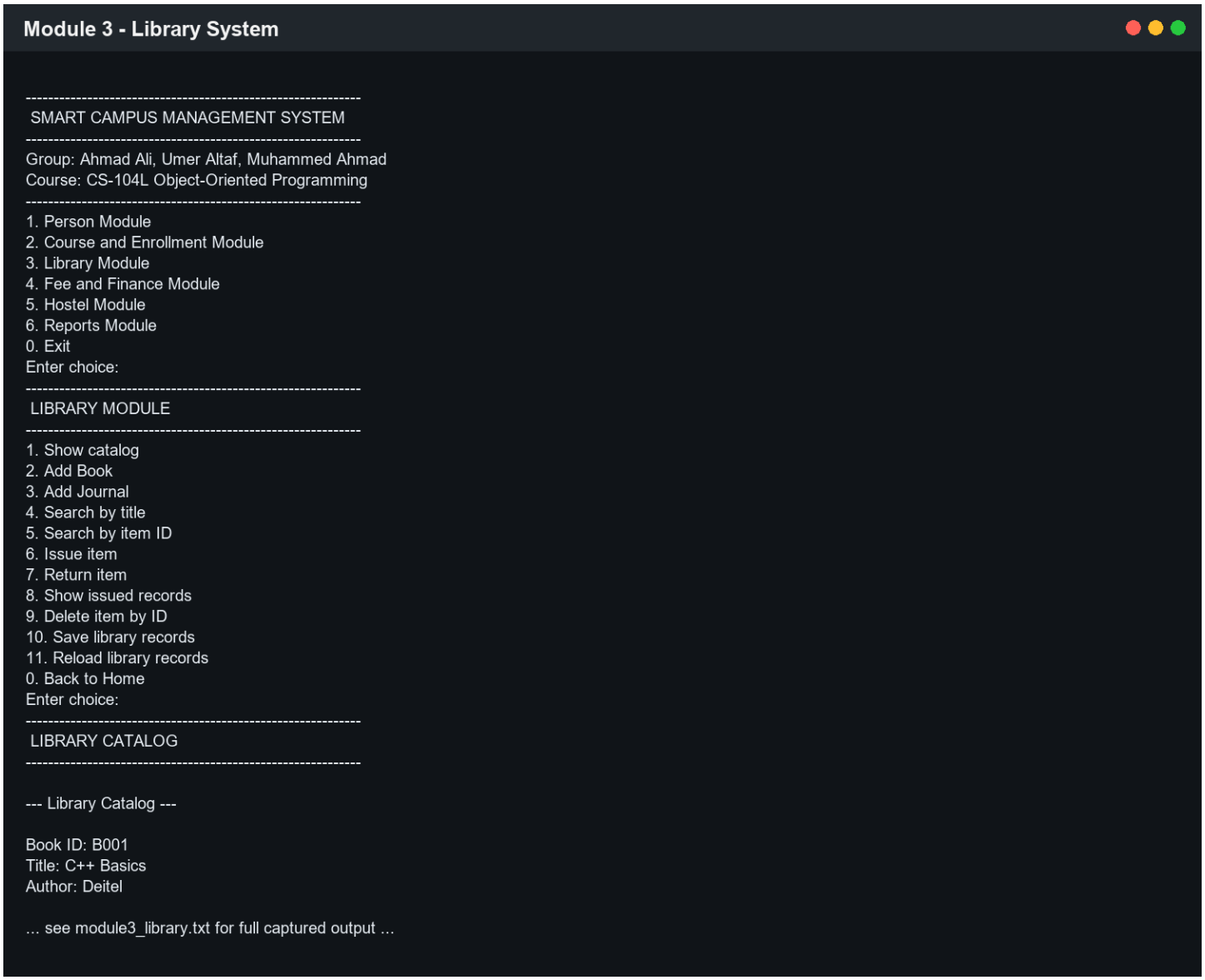
Module1 Add Course Test Output Screenshot



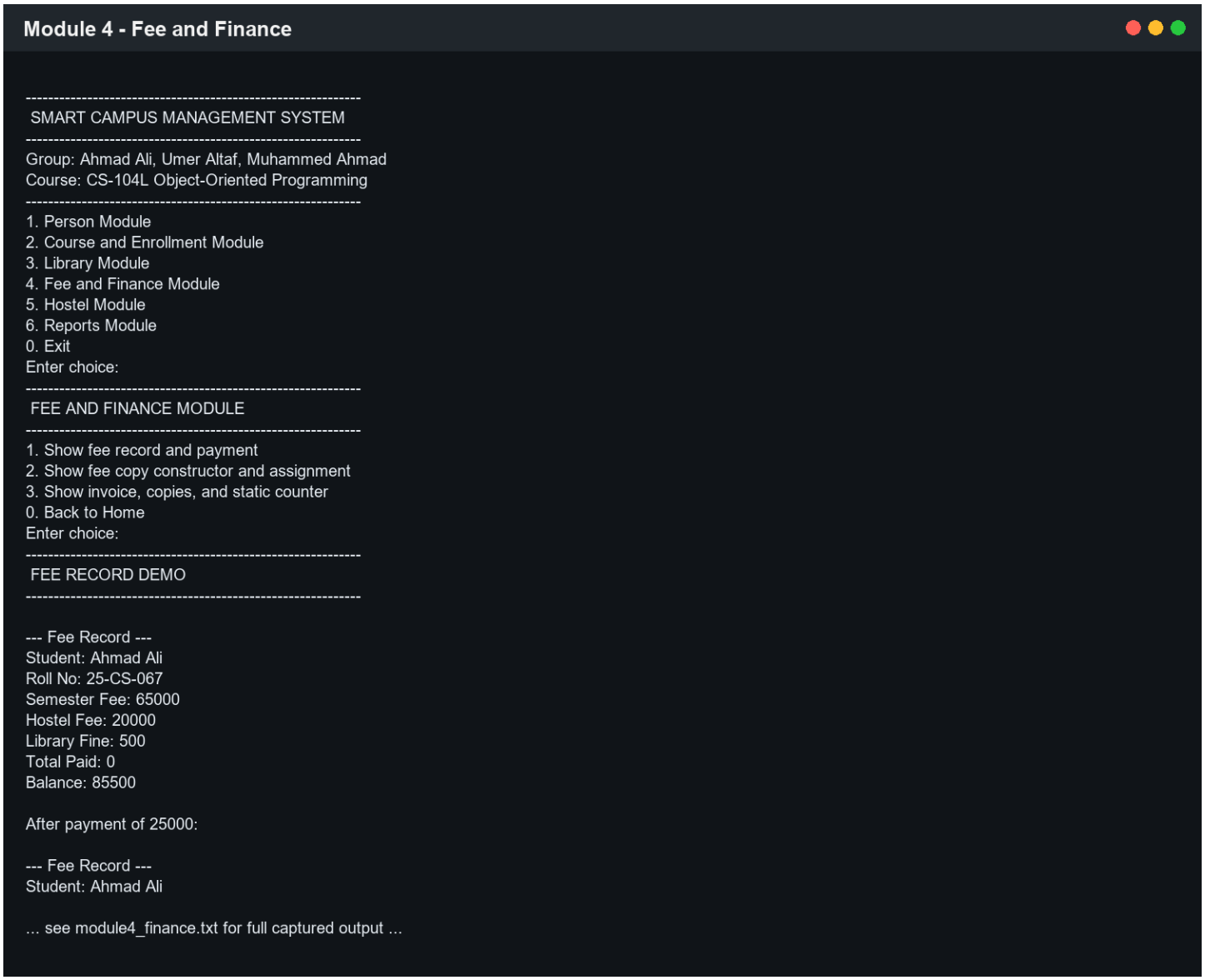
Module2 Course Output Screenshot



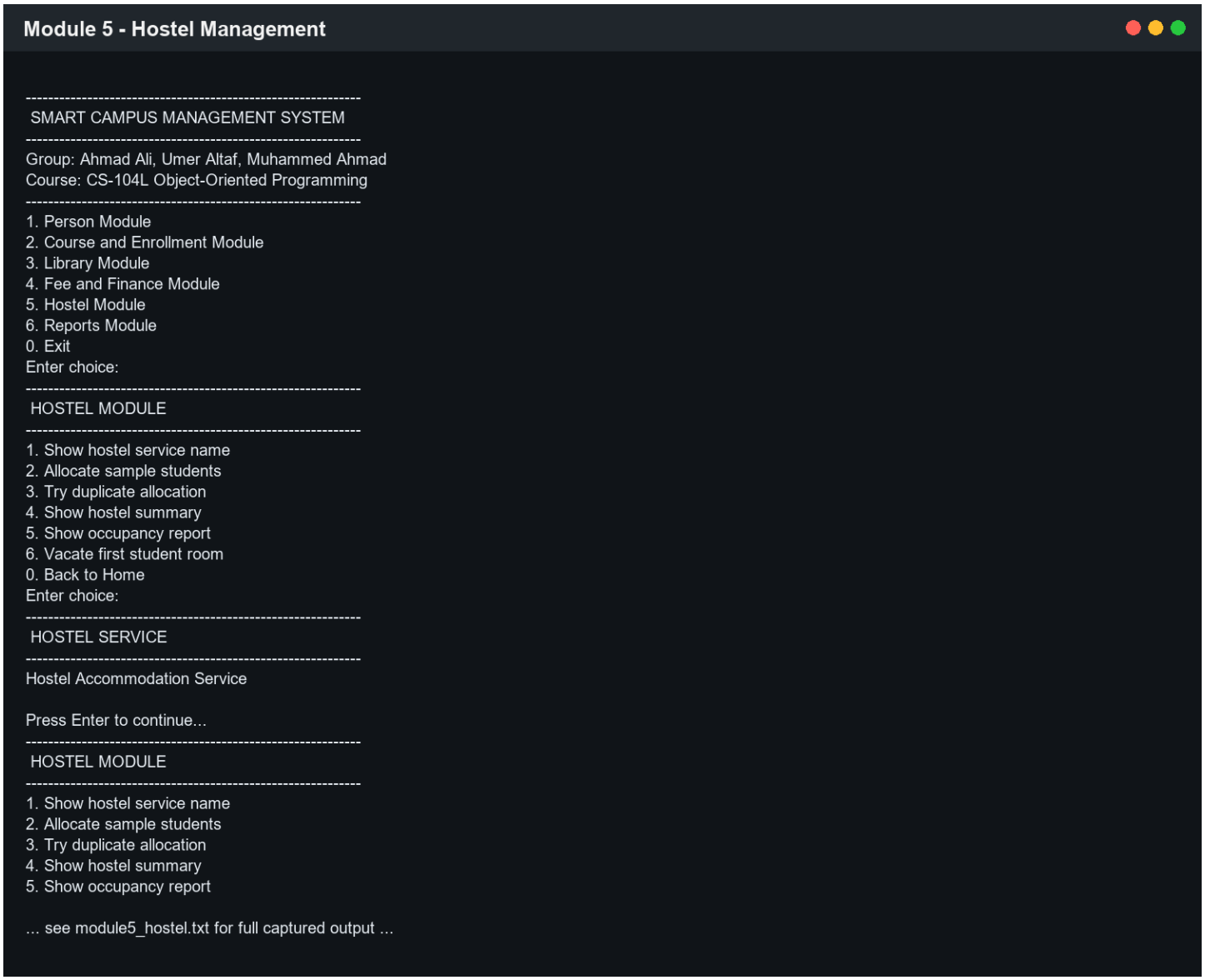
Module3 Library Output Screenshot



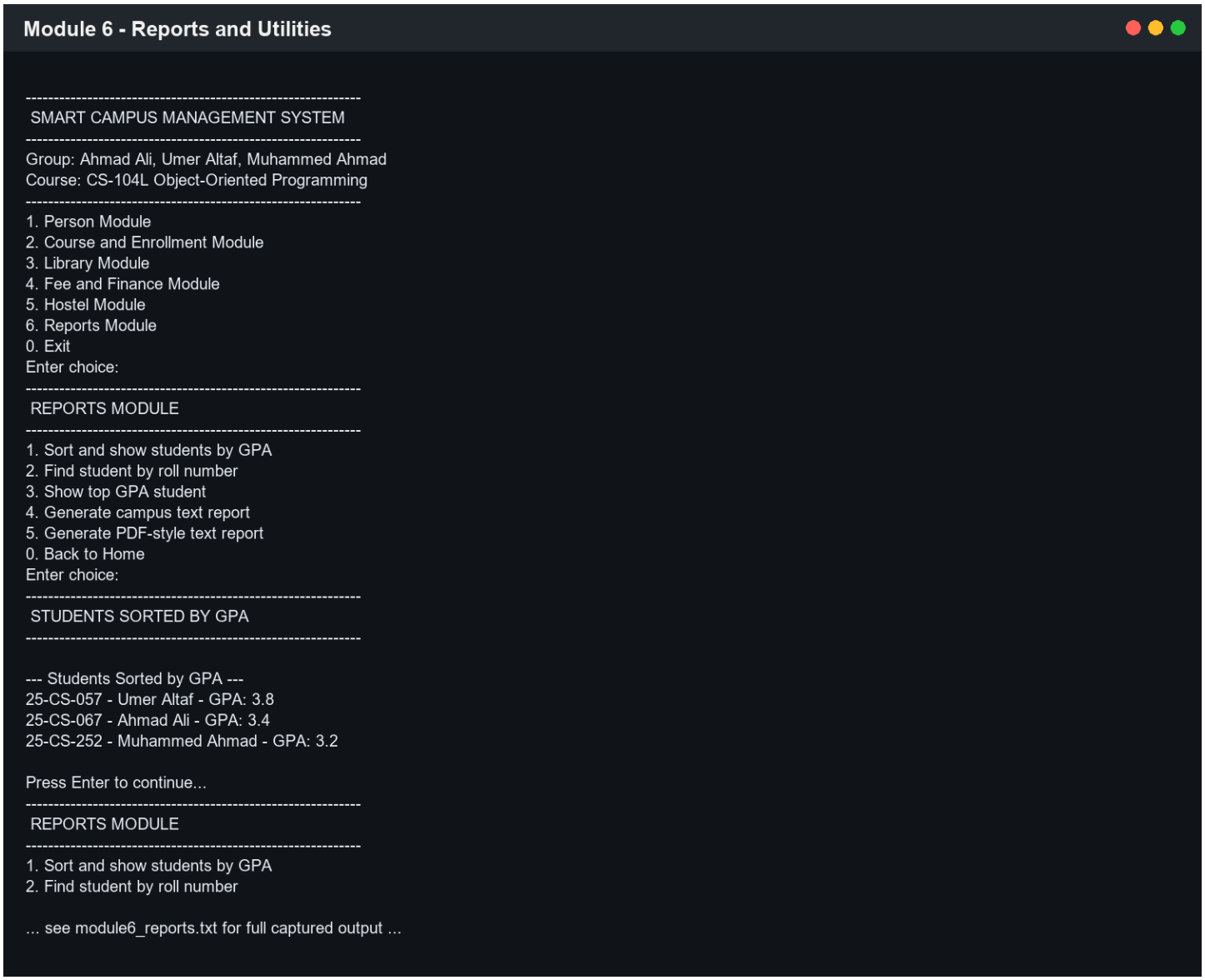
Module4 Finance Output Screenshot



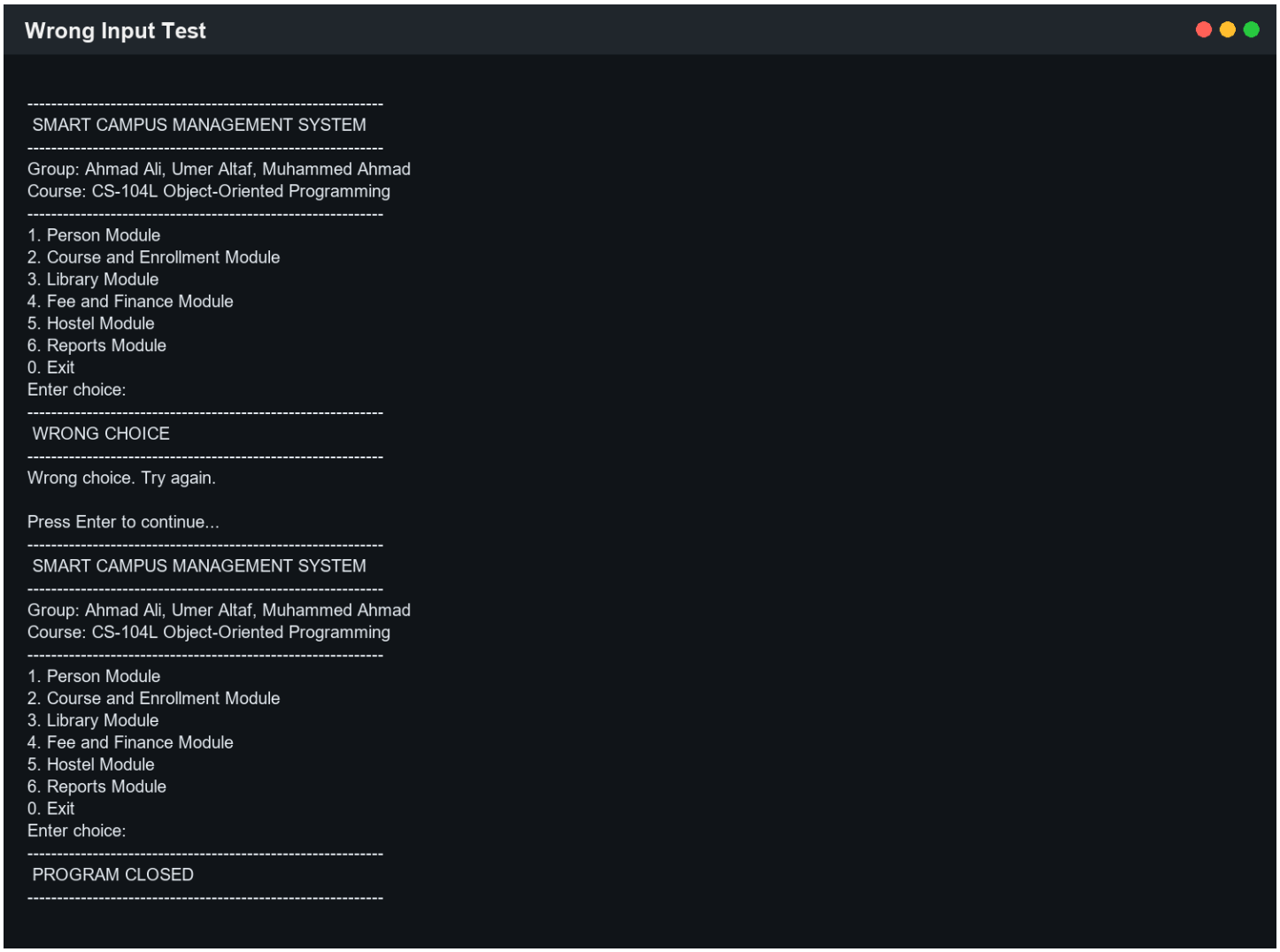
Module5 Hostel Output Screenshot



Module6 Reports Output Screenshot



Wrong Input Output Screenshot



Challenges and Solutions

- Understanding polymorphism: solved by using Person* array in main.cpp.
- Managing dynamic memory: solved by deleting Library items and Invoice item arrays.
- Handling full course capacity: solved by using a custom exception class.
- File handling format: solved by using simple pipe-separated text data.

Conclusion

This project helped practice core C++ OOP concepts in a practical campus system. The project is intentionally console-based and simple so each module can be explained in viva. Before final submission, the group should push the repository to GitHub and replace the placeholder GitHub URL in this report.

References

- Course project brief provided by HITEC University Taxila.
- C++ class notes and lab practice.
- cppreference.com for basic C++ syntax reference.